

Assignment 12.1 - Recurrent Neural Networks

Please submit your solution of this notebook in the Whiteboard at the corresponding Assignment entry as .ipynb-file and as .pdf.

Please state both names of your group members here:

Rashid Harvey and S M Shameem Ahmed Khan

Task 12.1.1: RNN - 'ShakesGen'

Let's create a ShakesGen !!

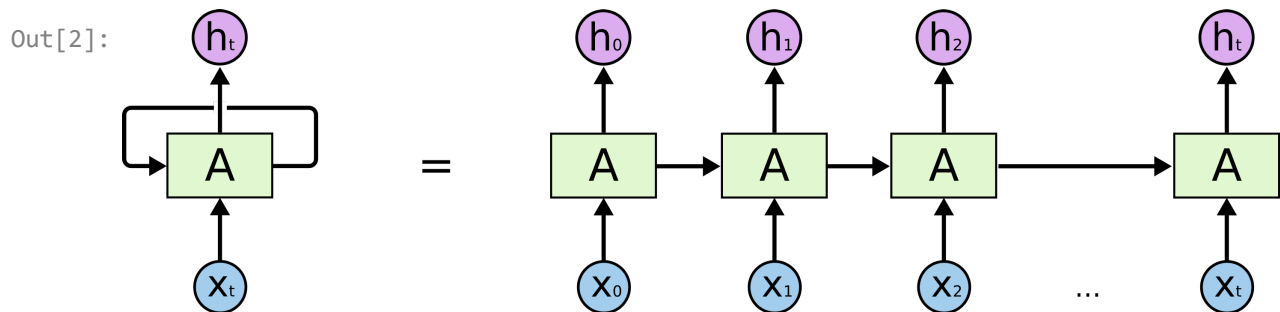
The data folder contains a shakespeare folder with works from William Shakespeare. Your task is to implement an RNN that learns to write Shakespeare-style text.

Below, you'll find all the utility code needed for this task. The Corpus class serves as a dataset, and you can retrieve a batch with its target by calling `get_batch` on a batchified dataset.

- Build the missing model components and train your ShakesGen model. **(RESULT)**
- Generate at least 30 lines of text using your ShakesGen model. **(RESULT)**

Especially, if you train on cpu, you can stop training after 5 minutes and generate based on the current model state.

```
In [2]: from IPython.display import Image
from IPython.core.display import HTML
Image(url= "https://miro.medium.com/max/4000/0*WdbXF_e8kZI1R5nQ.png", width=700)
```



```
In [3]: # Some imports
import torch
import torch.nn as nn
import torch.autograd as autograd
import torch.cuda as cuda
```

```

import torch.optim as optim
import torch.nn.functional as F
import os
import tqdm
import numpy as np

```

```

In [4]: class Dictionary(object):
        def __init__(self):
            self.word2idx = {}
            self.idx2word = []

        def add_word(self, word):
            if word not in self.word2idx:
                self.idx2word.append(word)
                self.word2idx[word] = len(self.idx2word) - 1
            return self.word2idx[word]

        def __len__(self):
            return len(self.idx2word)

class Corpus(object):
    def __init__(self, path):
        self.dictionary = Dictionary()

        # This is very english language specific
        # We will ingest only these characters:
        self.whitelist = [chr(i) for i in range(32, 127)]

        self.train = self.tokenize(os.path.join(path, 'train.txt'))
        self.valid = self.tokenize(os.path.join(path, 'valid.txt'))

    def tokenize(self, path):
        """Tokenizes a text file."""
        assert os.path.exists(path)
        # Add words to the dictionary
        with open(path, 'r', encoding="utf8") as f:
            tokens = 0
            for line in f:
                line = ''.join([c for c in line if c in self.whitelist])
                words = line.split() + ['<eos>']
                tokens += len(words)
                for word in words:
                    self.dictionary.add_word(word)

        # Tokenize file content
        with open(path, 'r', encoding="utf8") as f:
            ids = torch.LongTensor(tokens)
            token = 0
            for line in f:
                line = ''.join([c for c in line if c in self.whitelist])
                words = line.split() + ['<eos>']
                for word in words:
                    ids[token] = self.dictionary.word2idx[word]
                    token += 1

```

```

        return ids

def batchify(data, batch_size):
    # Work out how cleanly we can divide the dataset into bsz parts.
    nbatch = data.size(0) // batch_size
    # Trim off any extra elements that wouldn't cleanly fit (remainders).
    data = data.narrow(0, 0, nbatch * batch_size)
    # Evenly divide the data across the bsz batches.
    data = data.view(batch_size, -1).t().contiguous()
    return data

def get_batch(source, i, bptt_size=35):
    seq_len = min(bptt_size, len(source) - 1 - i)
    data = source[i:i+seq_len]
    target = source[i+1:i+1+seq_len].view(-1)
    return data, target

```

```

In [5]: # Use Corpus to Load data
        corpus = Corpus('./data/shakespeare')

```

```

In [6]: vocab_size = len(corpus.dictionary)
        print(vocab_size)

        # Print first 100 words from training data
        words = [corpus.dictionary.idx2word[corpus.train[i].item()] for i in range(min(100,
        print(' '.join(words))

```

74010

<eos> THE SONNETS <eos> <eos> 1 <eos> <eos> From fairest creatures we desire increas
e, <eos> That thereby beautys rose might never die, <eos> But as the ripper should by
time de cease, <eos> His tender heir might bear his memory: <eos> But thou contracted
to thine own bright eyes, <eos> Feedst thy lights flame with self-substantial fuel,
<eos> Making a famine where abundance lies, <eos> Thy self thy foe, to thy sweet sel
f too cruel: <eos> Thou that art now the worlds fresh ornament, <eos> And only heral
d to the gaudy spring, <eos> Within thine own bud buriest thy content, <eos>

```

In [7]: idx = corpus.dictionary.word2idx.get("THE", -1) # returns -1 if not found
        print(f"Index of the word 'THE': {idx}")

```

Index of the word 'THE': 1

```

In [8]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

In [13]: import time

        # inspired by https://gist.github.com/karpathy/d4dee566867f8291f086
        # bptt = backprop through time
        class ShakesGen:
            def __init__(self, corpus, batch_size=20, bptt_size=35, learning_rate=1e-1, hid
                self.corpus = corpus
                self.batch_size = batch_size
                self.bptt_size = bptt_size
                self.learning_rate = learning_rate
                self.hidden_size = hidden_size
                self.vocab_size = vocab_size

```

```

self.train_data = batchify(self.corpus.train, batch_size).to(device)
self.valid_data = batchify(self.corpus.valid, batch_size).to(device)

# parameters
self.W_xh = np.random.randn(hidden_size, vocab_size) * 0.01 # input to hidd
self.W_hh = np.random.randn(hidden_size, hidden_size) * 0.01 # hidden to hi
self.W_hy = np.random.randn(vocab_size, hidden_size) * 0.01 # hidden to out
self.bh = np.zeros((hidden_size, 1)) # hidden bias
self.by = np.zeros((vocab_size, 1)) # output bias

self.mW_xh = np.zeros_like(self.W_xh)
self.mW_hh = np.zeros_like(self.W_hh)
self.mW_hy = np.zeros_like(self.W_hy)
self.mbh = np.zeros_like(self.bh)
self.mby = np.zeros_like(self.by)

def get_train_batch(self, i):
    return get_batch(self.train_data, i, self.bptt_size)

def get_valid_batch(self, i):
    return get_batch(self.valid_data, i, self.bptt_size)

def one_hot(self, idx):
    x = np.zeros((self.vocab_size, 1))
    x[idx] = 1
    return x

# from karpathys blog
def step(self, inputs, targets):
    h, y, p = {}, {}, {}
    h[-1] = np.zeros((self.hidden_size, 1))
    loss = 0

    seq_len = len(inputs)

    # forward pass
    for t in range(seq_len):
        x = self.one_hot(inputs[t])
        h[t] = np.tanh(np.dot(self.W_xh, x) + np.dot(self.W_hh, h[t-1]) + self.
        y[t] = np.dot(self.W_hy, h[t]) + self.by # unnormalized log probabiliti
        p[t] = np.exp(y[t]) / np.sum(np.exp(y[t])) # probabilities for next cha
        loss += -np.log(p[t][targets[t], 0] + 1e-8) # softmax (cross-entropy lo

    # backward pass: compute gradients going backwards
    dW_xh = np.zeros_like(self.W_xh)
    dW_hh = np.zeros_like(self.W_hh)
    dW_hy = np.zeros_like(self.W_hy)
    dbh = np.zeros_like(self.bh)
    dby = np.zeros_like(self.by)
    dhnext = np.zeros_like(h[0])

    for t in reversed(range(seq_len)):
        x = self.one_hot(inputs[t])
        dy = np.copy(p[t])
        dy[targets[t]] -= 1 # backprop into y
        dW_hy += np.dot(dy, h[t].T)

```

```

        dby += dy
        dh = np.dot(self.W_hy.T, dy) + dhnext # backprop into h
        dhraw = (1 - h[t] * h[t]) * dh # backprop through tanh nonlinearity
        dbh += dhraw
        dW_xh += np.dot(dhraw, x.T)
        dW_hh += np.dot(dhraw, h[t-1].T)
        dhnext = np.dot(self.W_hh.T, dhraw)

    for dparam in [dW_xh, dW_hh, dW_hy, dbh, dby]:
        np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradien

    return loss, dW_xh, dW_hh, dW_hy, dbh, dby, h[seq_len-1]

def train(self, n_epochs=1, time_limit=300):
    start_time = time.time()

    for epoch in range(n_epochs):
        total_loss = 0
        n_batches = 0

        for i in range(0, self.train_data.size(0) - 1 - 1, self.bptt_size):
            if time.time() - start_time > time_limit:
                print(f"Time limit of {time_limit} seconds reached. Stopping tr
                return

            inputs, targets = self.get_train_batch(i)

            for b in range(self.batch_size):
                inp_seq = inputs[:, b].cpu().numpy()
                tgt_seq = targets.view(inputs.size(0), -1)[:, b].cpu().numpy()

                loss, dW_xh, dW_hh, dW_hy, dbh, dby, _ = self.step(inp_seq, tgt
                total_loss += loss

                # perform parameter update with Adagrad
                self.mW_xh += dW_xh * dW_xh
                self.mW_hh += dW_hh * dW_hh
                self.mW_hy += dW_hy * dW_hy
                self.mbh += dbh * dbh
                self.mby += dby * dby

                self.W_xh -= self.learning_rate * dW_xh / np.sqrt(self.mW_xh +
                self.W_hh -= self.learning_rate * dW_hh / np.sqrt(self.mW_hh +
                self.W_hy -= self.learning_rate * dW_hy / np.sqrt(self.mW_hy +
                self.bh -= self.learning_rate * dbh / np.sqrt(self.mbh + 1e-8)
                self.by -= self.learning_rate * dby / np.sqrt(self.mby + 1e-8)

            n_batches += 1

        avg_loss = total_loss / (n_batches * self.batch_size)
        elapsed = time.time() - start_time
        print(f"Epoch {epoch+1}, Avg Loss: {avg_loss:.4f}, Time: {elapsed:.1f}s

def sample(self, seed_ix, n):
    h = np.zeros((self.hidden_size, 1))
    x = self.one_hot(seed_ix)

```

```

ixes = []

for t in range(n):
    h = np.tanh(np.dot(self.W_xh, x) + np.dot(self.W_hh, h) + self.bh)
    y = np.dot(self.W_hy, h) + self.by
    p = np.exp(y) / np.sum(np.exp(y))
    ix = np.random.choice(range(self.vocab_size), p=p.ravel())
    x = self.one_hot(ix)
    ixes.append(ix)

return ixes

```

```

In [15]: shakesgen = ShakesGen(corpus)

shakesgen.train(n_epochs=10, time_limit=900)

seed_ix = corpus.dictionary.word2idx.get("THE", 0)
sample_indices = shakesgen.sample(seed_ix, 200)
sample_words = [corpus.dictionary.idx2word[ix] for ix in sample_indices]

output_text = ' '.join(sample_words)
lines = []
current_line = []
for word in sample_words:
    current_line.append(word)
    if word == '<eos>':
        lines.append(' '.join(current_line))
        current_line = []
        if len(lines) >= 30:
            break

if len(lines) < 30 and current_line:
    lines.append(' '.join(current_line))

print("Generated Shakespeare-style text (30+ lines):")
print("-" * 50)
for i, line in enumerate(lines, 1):
    print(f"{i}: {line.replace('<eos>', '')}")

```

Time limit of 900 seconds reached. Stopping training.

Generated Shakespeare-style text (30+ lines):

1: bedfellow,
2: a were me, queen. at can thy
3: of hey, was the in not fancy's To ho, a No, Kept
4: in to the I provision, with
5: ho,
6: death. is you lawless whoreson fancy's and
7: MERCHANT. upon flag fancy's hath sings the be had the Boy, sings the That is the
make you With a should the you, sings the in ho, MERCHANT. doth sings tiny
8:
9: habit, her And a
10: my
11: I'll Englishman? the hey, cling
12: harvest, eyes in whoreson proceed. approach. me ho, being you, ho,
13:
14: ho, the her is word FALSTAFF. I it What Orsino's beer? Enter in
15: and
16: newly breaths ho, the hath
17: that thy my world boy, at for PAINTER, Apothecary!
18: That of tiny KING ho,
19:
20: sings of our up
21:
22:
23: your queen. Take centre, how?- confess,
24: my for fancy's
25: sings the and sings fancy's her ho, gives didst ho, being you? at of poor that t
he queen. the Greece
26: When With her my fancy's this cut the in the
27:
28: Tell you for How heavens in give
29: jewel- in the

Congratz, you made it! :)